Towards a WasmGC backend and runtime for the MicroHs compiler

Master's thesis part 1: project proposal

Martijn Voorwinden – 7854064
Utrecht University – Department of Information and Computing Sciences
1st supervisor: Marco Vassena
2nd supervisor: Wouter Swierstra
Daily supervisor: Apoorva Anand

Abstract

Today, porting Haskell code to web browsers and cloud platforms is complicated. Current approaches such as transpilation to JavaScript or WasmMVP suffer from issues such as large binary sizes or performance issues. To solve this issue, we explore the use of WasmGC as a target for Haskell-to-Wasm compilers. We do this by extending the MicroHs compiler with a new backend that targets WasmGC and a runtime system that goes with it. The runtime system consists of a graph reduction machine capable of reducing SKI combinators, Turner's combinators, and a set of strict primitives. To help in the implementation of the runtime system, we design and implement a Haskell-based DSL for generating WasmGC code. We evalutate the generated Wasm binaries in terms of size and runtime performance against the existing MicroHs Wasm backend (through emscripten). Furthermore, we discuss the plan going forward for Part 2 of the thesis.

Contents

1 Introduction **1**

2 Background **3**
2.1 MicroHs 3
2.2 WebAssembly 6

3 Compiler and Runtime System Design **9**
3.1 Compiler and Runtime System Overview 9
3.2 Domain Specific Language 9
3.3 WasmGC Backend Implementation 11
3.4 Runtime System Implementation 11

4 Preliminary Results **15**

5 Related Work **17**
5.1 Haskell for the Web 17
5.2 WebAssembly with Garbage Collection 17
5.3 Combinator Based Compilers and Runtime Systems 18

6 Plan **19**

Bibliography **22**

Appendices **23**
A: Ethics and Privacy QuickScan 24

Introduction

Today, porting Haskell code to web browsers and cloud platforms is complicated. Current approaches such as transpilation to JavaScript [1] suffer from performance issues and lack of semantic similarity (e.g. functional vs imperative, lazy vs eager eveluation, etc.), while solutions such as containerization can not be used in browser environments.

A promising alternative is compiling Haskell to WebAssembly (Wasm) [2, 3, 4], a low-level bytecode format designed for efficient execution in portable environments such as web browsers, but also for non-browser engines which can be used for server-side applications. Wasm however has its own challenges, the original version of Wasm (WasmMVP) does not natively support garbage collection (GC), which forces compilers targeting WasmMVP to ship their own memory management systems. This in turn results in large binary sizes, which causes issues on the web where bundle size is the main contributor to loading time.

This changed with the recent release of Wasm 3.0 [5] due to the integration of the WasmGC proposal, which natively supports GC by leveraging the host environment's GC capabilities. This allows compilers targeting Wasm to make use of the host environment's GC instead of shipping their own, which can lead to smaller binary sizes and improved performance.

Despite the potential benefits of using WasmGC, current Haskell-to-Wasm compilers such as GHC's Wasm backend [3] and Asterius [4] do not make use of it, and instead target WasmMVP.

Contributions We aim to explore the effects of using WasmGC as a target for Haskell-to-Wasm compilers, which we will do by extending an existing Haskell compiler MicroHs [6] to target WasmGC. Concretely, we will do the following:

1. Design and implement a custom domain specific language (DSL) for generating WasmGC code, for use in the runtime system and backend of the compiler. This will help us to write more maintainable and readable code compared to writing raw WasmGC.

2. Design and implement a runtime system for the WasmGC backend using the custom DSL.

3. Evaluate the produced Wasm binaries in terms of size and runtime performance against other Haskell-to-Wasm compilers and MicroHs's existing C backend.

We expect that using WasmGC will lead to smaller binary sizes due to a smaller runtime system (without a custom GC) and improved performance due WasmGC specific optimizations in Wasm engines and optimizers.

Previous Work Previously, Anand [7] made a prototype extension to MicroHs to target WasmGC and implemented a proof-of-concept runtime system written in a combination of WasmGC and a custom DSL. Initial experiments showed promising results in terms of binary size, however runtime performance was not tested. Our work will extend this work with a new DSL capable of generating the full runtime system instead of just parts of it, building a more complete runtime system that supports more Haskell features, and evaluating the runtime performance of the generated Wasm binaries.

Document Structure The rest of this thesis proposal is structured as follows:

- Chapter 2 provides the reader with the necessary background on WebAssembly and the architecture of MicroHs.
- Chapter 3 describes the design and implementation of the WasmGC backend and runtime system.
- Chapter 4 describes preliminary results on the runtime performance of the generated Wasm binaries.
- Chapter 5 discusses related work on the topic.
- Chapter 6 outlines the plan for Part 2 of the thesis.

Background

2.1 MicroHs

MicroHs is a small Haskell compiler that has a relatively standard compilation pipeline. The compiler starts by parsing the Haskell source code into an abstract syntax tree (AST). The AST is then type-checked and subsequently desugared into lambda terms. The lambda terms are optimized and then transformed into combinators through the use of a process called bracket abstraction. After which the abstract representation of the combinators is converted into a useable format for the runtime written in C. When this is done, the runtime system and combinators are combined into a single C codebase, which is then either compiled to a native executable using a standard C compiler or converted to WebAssembly using Emscripten We are mostly interested in the final stages of the pipeline. The process is illustrated in Figure 2.1. The compiler is bootstrapped, meaning that it is written in Haskell and compiles itself. The source code for the compiler can be found on GitHub [8].

Combinators MicroHs uses a combination of SKI combinators, an extended set of combinators, and primitive operations (intrinsics for arithmetic, I/O, etc.) to represent Haskell code. SKI combinators are a small set of combinators that can be used to represent any lambda term. The three basic SKI combinators are:

$$
\begin{aligned}
S\ x\ y\ z &= x\ z\ (y\ z) \\
K\ x\ y &= x \\
I\ x &= x
\end{aligned}
$$

Further combinators include those used by Turner [9] or are added by MicroHs itself to improve performance of the generated code. The reduction rules for these combinators

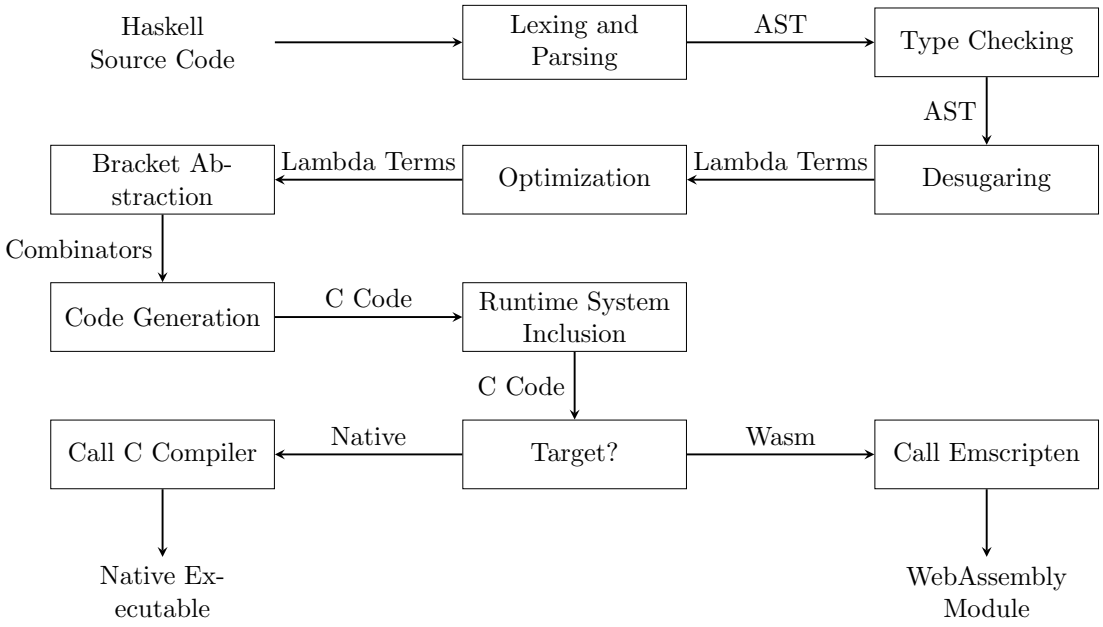

Figure 2.1: MicroHs Compilation Pipeline

are as follows:

$$
\begin{aligned}
B\ x\ y\ z &= x\ (y\ z) \\
C\ x\ y\ z &= x\ z\ y \\
S'\ x\ y\ z\ w &= x\ (y\ w)\ (z\ w) \\
B'\ x\ y\ z\ w &= x\ y\ (z\ w) \\
C'\ x\ y\ z\ w &= x\ (y\ w)\ z \\
A\ x\ y &= y \\
U\ x\ y &= y\ x \\
Z\ x\ y\ z &= x\ y \\
P\ x\ y\ z &= z\ x\ y \\
R\ x\ y\ z &= y\ z\ x \\
O\ x\ y\ z\ w &= w\ x\ y \\
Y\ x &= x\ (Y\ x) \\
K2\ x\ y\ z &= x \\
K3\ x\ y\ z\ w &= x \\
K4\ x\ y\ z\ w\ v &= x
\end{aligned}
$$

Representing Combinators To efficiently represent combinators and their arguments, MicroHs uses a graph representation. Each node can either be a leave node storing a combinator or a primitive value, or an internal node storing a reference to its left and right children. This allows for efficient sharing of subterms and avoids the need for copying when reducing combinators. For example, the combinator application `K (I x) y` can be represented as the graph shown in Figure 2.2.

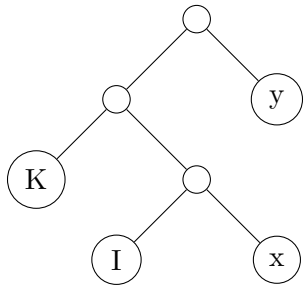

Figure 2.2: Graph representation of the combinator application `K (I x) y`

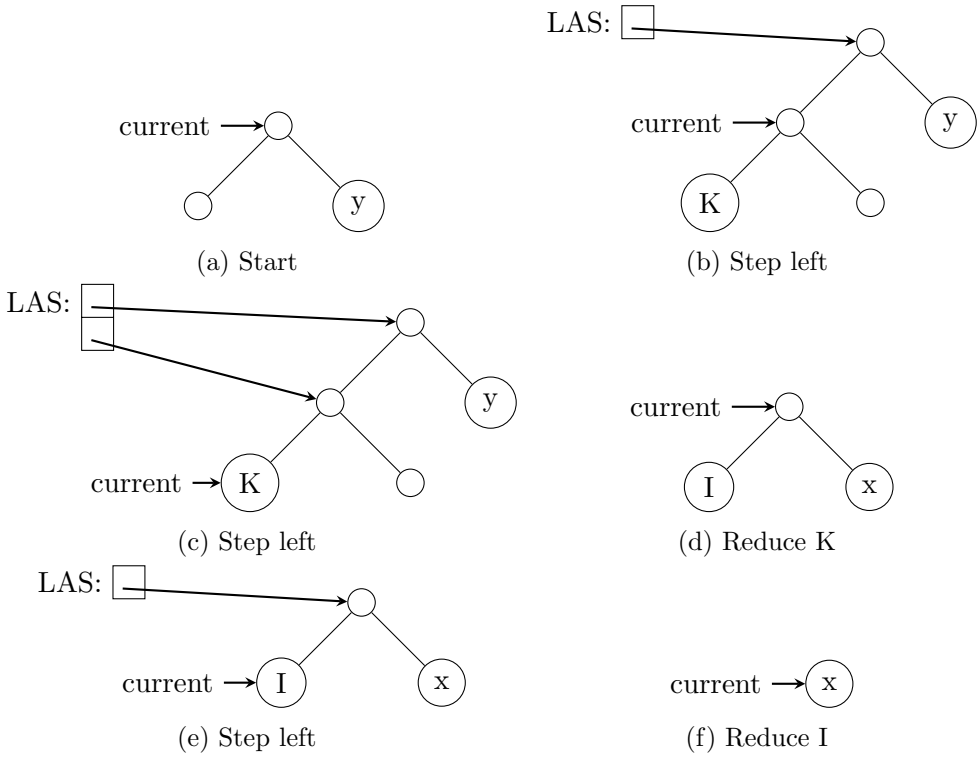

Figure 2.3: Reduction of the combinator application `K (I x) y`

Runtime System After building the graph representation of the combinators, the runtime system is responsible for reducing the graph to weak head normal form (WHNF). This is done by repeatadly walking the left spine of the graph until a leaf storing a combinator is reached, at which point the reduction rules for that combinator are applied. Since the reduction rules for most combinators involve multiple arguments, the runtime system maintains a left ancestor stack (LAS) to keep track of the ancestors of the current node (the left spine of the graph). This allows the runtime system to efficiently access the arguments needed for reduction without having to traverse the graph multiple times. An example reduction for the combinator application `K (I x) y` is shown in Figure 2.3.

Another important thing to note about the runtime system is the existence of strict primitives, such as arithmetic operations, which require all of their arguments to be fully reduced before they can be applied. This means that when the runtime system encounters a strict primitive, it must first reduce all of its arguments to WHNF before applying the primitive operation and continuing with the reduction of the resulting graph.

2.2 WebAssembly

Wasm is a binary stack-based instruction format. Wasm programs are ran inside a host environment that can influence the wasm code through imports and exports. The programs themselves are organized into modules containing functions, memory, imports/exports, and other components.

Wasm has a text format called WAT (WebAssembly Text format) which is a human-readable representation of the binary format. From now on we will use WAT syntax to illustrate Wasm concepts. Using WAT a simple list of instructions that adds two numbers looks like this:

```
(i32.const 1)
(i32.const 2)
(i32.add)
```

The first two instructions push the constants 1 and 2 onto the stack, and the last instruction pops the two values from the stack, adds them together, and pushes the result (3) back onto the stack.

Instructions Instructions in Wasm are the basic building blocks of computation. They can be broadly categorized into two types: control flow instructions and simple instructions.

Simple instructions perform basic operations such as arithmetic, logical operations, and memory access. Examples include `i32.add` for adding two 32-bit integers, `i32.sub` for subtraction, and `local.get` and `local.set` for accessing local variables.

Control flow instructions manage the flow of execution within a function. Examples include `if`, `block`, `loop`, and `br` (branch). These instructions allow for conditional execution, loops, and jumps within the function body. Since Wasm does not have boolean types, conditions are represented using integers, where 0 is false and any non-zero value is true. When branching to a label belonging to a block, the execution continues after the end of the block. A loop label, on the other hand, causes execution to continue at the beginning of the loop. An example of the control flow instructions in action is shown below:

```
(block $exit
  (loop $loop
    ;; check if counter == 0
    (local.get $counter)
    (i32.const 0)
    (i32.eq)
    (br_if $exit) ;; if counter == 0, exit block
    ;; decrease counter
    (local.get $counter)
    (i32.const 1)
    (i32.sub)
    (local.set $counter)
    (br $loop) ;; continue looping
  )
)
```

Functions Functions in Wasm consist of zero or more parameters, results, and local variables, and a body containing instructions. A simple function that adds two numbers and doubles the result can be defined like this:

```
(func $addAndDouble (param $a i32) (param $b i32) (result i32)
  (local $sum i32)
  (local.get $a)
  (local.get $b)
  (i32.add)
  (local.set $sum)
  (local.get $sum)
  (local.get $sum)
  (i32.add)
)
```

In this example the function takes two parameters of type `i32` and returns a single `i32` result. The body of the function retrieves the parameters from the stack, adds them together, stores the result in a local variable, and then doubles the result by adding it to itself before returning it (a return is done by leaving the result on the stack at the end of the function body).

Functions can be called using the `call` instruction, which takes the function identifier as an argument and expects the appropriate number of arguments to be on the stack:

```
(i32.const 1)
(i32.const 2)
(call $add)
```

Types Wasm has a couple of different categories of types, the most basic ones are the primitive numeric types, these include `i32` (32-bit integer), `i64` (64-bit integer), `f32` (32-bit float), and `f64` (64-bit float), as well as `v128` (128-bit SIMD vector).

We can also box types through the use of references (note that references are explicitly typed in Wasm, meaning that every reference has a specific type associated with it), these have special instructions for manipulating them: `ref.null`, `ref.is_null`, `ref.cast`, `ref.test`, and `ref.eq`. `ref.null` creates a null reference of a given type, `ref.is_null` checks if a reference is null, `ref.cast` casts a reference to a different type (and traps if the cast is invalid), `ref.test` checks if a reference can be cast to a given type, and `ref.eq` compares two references.

These reference types can be used for boxing numeric types, but are most oftenly used for more complex types named aggregate types. Aggregate types include Structs and Arrays, and are user defined. Structs are collections of named fields, each with their own type, while Arrays are collections of elements of the same type. An example of type definitions for node which we could use for the implementation of the MicroHs runtime system is shown below:

```
(type $Node (struct
  (field $left  (mut anyref))
  (field $right (mut anyref))
))
```

In this definition, we use the `anyref` type which is a supertype of all reference types in Wasm. This allows us to store references to any type of object in the `left` and `right`

fields of the `Node` struct, but comes at the cost of having to perform runtime type casts when accessing these fields. The `mut` keyword indicates that the fields are mutable, meaning that their values can be changed after the struct is created. Simlarly we can define an array type like this:

```
(type $NodeArray (array (mut (ref null $Node))))
```

Memory Originally Wasm provided only one type of memory, namely linear memory, which is a contiguous, byte-addressable array of memory that can be read and written to from Wasm code. Linear memory is managed manually by the program, meaning that the program is, for example, responsible for not overwriting memory that is still needed, and is thus prone to memory issues caused by human error.

Since the release of WasmGC, Wasm now also supports garbage-collected (GC) memory, it does this by leveraging the host environment's garbage collector reclaim and allocate memory on behalf of the Wasm program. Objects pointed to by references are automatically stored in GC memory.

Compiler and Runtime System Design

3.1 Compiler and Runtime System Overview

If you recall the architecture of the MicroHs compiler from Chapter 2, the compiler transforms the generated combinators to a useable format for the C runtime, after which it combines the generated code with the runtime system and compiles it to a binary.

To target WasmGC, we will need to change these two steps. Instead of generating the combinator format that the C runtime can use, we will instead generate WasmGC code directly from the combinators. As for the runtime system, we will need to reimplement it in WasmGC. This means that we will implement the graph reduction algorithm in WasmGC, to do this we will make use of the algorithm shown in Algorithm 1. This algorithm takes a node as input and reduces it to WHNF by repeatedly walking the left spine of the graph until a leaf storing a combinator or primitive is reached, at which point we call a function `step` that applies the reduction rule for that combinator or primitive and returns a pointer to the reduced node and an updated LAS pointer. The runtime system also maintains a left ancestor stack (LAS) to keep track of the ancestors of the current node and a LAS pointer to point to the top of the LAS.

Algorithm 1: Graph reduction algorithm

```
Input  : Node n to reduce
Output: Reduced node
las ← ∅;
las_pointer ← −1;
cur ← n;
while cur is not fully reduced do
    if cur is a combinator then
        (cur, las_pointer) ← step(cur, las, las_pointer);
    else
        cur ← leftChild(cur);
    end
    las[las_pointer] ← cur;
    las_pointer ← las_pointer + 1;
end
```

3.2 Domain Specific Language

To help in generating WasmGC code for both the runtime system and WasmGC backend, we have designed and implemented a small DSL embedded in Haskell. The DSL is a deeply embedded imperative language [10] that abstracts away the explicit stack

manipulation that is necessary when writing Wasm code directly. A module in Wasm corresponds to a `RtModule` in the DSL, and contains the type definitions, imports, and functions of the module:

```
data RtModule = RtModule
    { moduleTypes   :: [RtTypeDef]
                    --  Name     Args     Results
    , moduleImports :: [(String, [RtTyp], [RtTyp])]
    , moduleFuncs   :: [RtFunc]
    , moduleMain    :: RtMainFunc
    }
```

The type definitions consist of a name and what type to bind it to:

```
data RtTypeDef =  RtTypDef
    { typeName :: String
    , typeDef  :: RtTyp
    }
```

Types are either a simple primitive type, a reference to another type, an array, or a record:

```
data  RtTyp
    = TI32
    | TBool
    | TAny
    -- any other wasm primitive type identified by its name, e.g. i64, f32, etc.
    | TOther  String
    -- a reference to another type, identified by its name.
    | TNRef   String
    | TRef    RtTyp
    | TArray  RtTyp
    | TRecord [(String, RtTyp)]
```

Like Wasm, functions have explicitly declared local variables at the function definition level, they have a name, an explicit type, and a body consisting of a single expression:

```
data RtFunc = RtFunc
    { funcName   :: String
    , funcArgs   :: [(String, RtTyp)]
    , funcRetTyp :: [RtTyp]
    , funcLocals :: [(String, RtTyp)]
    , funcBody   :: RtExpr
    }
```

The body of a function consists of a single expression `RtExpr` that can be a block of expressions, a control flow construct, or a simple expression:

```
data RtExpr
    = Block [RtExpr]
    | If     RtExpr  RtExpr RtExpr
    | While  RtExpr  RtExpr
    | Case   RtExpr [RtExpr]
```

```
    | Call   String [RtExpr]
    | Let    String  RtExpr
    | Return RtExpr
    ...
    | IntLit Int
```

Overall the DSL closely resembles WasmGC. Examples of the DSL in use can be found in the next section where we describe the runtime system implementation. For the examples we will make use of some helper functions that generate common patterns in the code, such as `mkNode` which generates the code to construct a new node with two children, `mkValue` which generates the code to box a literal in a `value` type, and `intRefLit` which takes an integer literal and converts it to a `i31` reference.

3.3 WasmGC Backend Implementation

The abstract representation of expressions in the MicroHs compiler is defined as follows:

```
data Exp
    = Var String
    | Lit Lit
    | App Exp Exp
    | Lam Ident Exp
```

To generate WasmGC code from this representation, we will need to map the abstract representation to the correct code to construct a graph in our DSL. If we encounter a variable, we generate the instructions to lookup the variable in the functions array and return a pointer to the variable in question. In case of a literal, we either generate instructions to box the literal in a `value` type (more on this in the next section) or if the literal is a combinator, we look it up in the combinator table and retrieve the index of the combinator, which is returned (as it is used an identifier). Applications are done by constructing a new node with the two arguments as its children. Lambdas should be eliminated by the time we reach the WasmGC backend, so we can ignore them. The code for this mapping is as follows:

```
compileExp :: Exp -> RtExpr
compileExp (Var x)          = AGet (DSL.Var _functions) (IntLit tIdx) _stack
    where tIdx = read (tail (showIdent v)) :: Int
compileExp (Lit (LPrim p)) = intRefLit $ case primIndex p of
    Just idx -> idx
    Nothing  -> error "Unknown primitive"
compileExp (Lit (Lint i))  = mkValue $ intRefLit i
compileExp (App e1 e2)     = mkNode (compileExp e1) (compileExp e2)
compileExp (Lam _ _)       = error "Lambdas should not exist"
```

3.4 Runtime System Implementation

The runtime system implements the algorithm shown in Algorithm 1 in WasmGC using our DSL. To do this, we first need to define the data structures we will use for the graph representation and the LAS. For the graph representation, we will use a struct type named `Node` with two fields `left` and `right` which are references to either other nodes, combinators/primitives, or values (we make use of `anyref` for this as it can store

references to any type). For the LAS, we will use an array of references to nodes, and lastly for values we will use a struct type named `Value` with a field `val` of type `anyref` to store primitive or fully reduced values. The type definitions for these types are as follows:

```
node = RtTypDef "Node" $ TRecord
    [ ("left",  TRef (TRef TAny))
    , ("right", TRef (TRef TAny))
    ]

las = RtTypDef "Stack" $ TArray (TNRef "Node")

value = RtTypDef "Value" $ TRecord
    [ ("val", TRef TAny) ]
```

We then require three functions, namely `main` which serves as the entry point for a program and constructs the initial graph and starts the reduction process, `reduce` which implements the graph reduction algorithm, and `step` which applies the reduction rule for a given combinator or primitive. The body of the `main` function is as follows:

```
mainBody fs = Block
    [ -- initialize temporary functions array
      Let ["functions"] (ANew (IntLit fs) "Stack")
    , -- fill the functions array with empty nodes
      While (ILt (Var "i") (IntLit fs))
        (Block
            [ ASet (Var "functions") (Var "i") (...) "Stack"
            , Let ["i"] (IAdd (Var "i") (IntLit 1))
            ]
        )
    , -- code for constructing the initial graph is inserted here by the backend
      -- root node is stored in a local variable named "node"
       ...
    , -- free the memory used by the functions array
      Let ["functions"] (NullLit (TNRef "Stack"))
    , -- start the reduction process
      Return (Call "reduce" [Var "node"])
    ]
```

The `reduce` function starts by initializing an array to be used as the LAS and a pointer to keep track of the top of the LAS. It then enters a loop where there are four cases to consider. Case 1: if the current node is `null`, then we have finished reduction and there was no result. Case 2: if the current node has a left child that is the `I` combinator and a right child of the type value, then we have finished reduction and the result is the value stored in the right child. Case 3: if the current node has a left child that is a combinator or primitive, then we call the `step` function to apply the reduction rule for that combinator or primitive. Case 4: if the current node has a left child that is not a combinator or primitive, then we walk down the left spine of the graph by setting the current node to its left child and pushing it onto the LAS. The code for this is as follows:

```
reduceBody = Block
      -- setup
    [ Let ["stack"]   (ANew (IntLit 100000) "Stack")]
    , Let ["sptr"]    (IntLit (-1))
      -- root is the argument to the function
    , Let ["current"] (Var "root")
      -- reduction loop
    , While Nop -- we handle exiting the loop using return
        (Block
            [ -- case 1:
              If (IsNull (Var "current"))
                 (Return (NullLit (TNRef "Value")))
                 Nop
              -- case 2:
              , If (TTest (getLeft  (Var "current")) (TRef TI32))
                   (If (IAnd ...)
                       (Return (...))
                       Nop
                   )
              Nop
              -- non terminating case,
              -- so we push the current node onto the LAS
            , Let ["sptr"] (increment $ Var "sptr")
            , ASet (Var "stack") (Var "sptr") (Var "current") "Stack"
            , If (TTest (getLeft (Var "current")) (TRef TInt))
                 -- case 3:
                 (Let ["current", "sptr"]
                    (Call _step [...]))
                 -- case 4:
                 (Let ["current"]
                    (TCast (getLeft (Var "current")) (TNRef "Node")))
            ]
        )
    , Return (NullLit (TNRef "Value")) -- should never reach here
    ]
```

The `step` function has a case for each combinator and primitive, which apply the reduction rule for the combinator/primitive in question. For this is makes use of metavariables to represent arguments extracted from the LAS, this makes the reduction rules look closer to the mathematical definitions of the combinators and primitives. The code for the `step` function is as follows:

```
stepBody = Block
    [ -- extract the combinator/primitive from the current node
      Let [_comb]
        (TRefInt $ RGet (Var _node) _left _Node)
    , Case (Var _comb) prims
    , Return $ Block [ Var _current, Var _i ]
    ]
```

It makes use of a helper function `prims` that generates the cases for each combina-

tor/primitive based on a primitive table that contains the reduction rules for each combinator/primitive. These reduction rules are defined using a helper function `redRule` which takes a left node definition, a right node definition, and the amount of arguments to pop from the LAS. Using this helper function, we can easily define reduction rules such as the following rule for the `B` combinator $B\ x\ y\ z = x\ (y\ z)$:

```
redRuleB = redRule ln rn n
    where
        ln = x
        rn = mkNode y z
        n  = 3
```

Preliminary Results

In order to determine if the direction we have taken with the WasmGC backend and runtime system is promising (i.e. if the generated Wasm binaries have reasonable runtime performance). we ran a small benchmark program on both the WasmGC backend as well as the existing Wasm backend (through emscripten) and compared the results. To force the execution of the program, we implemented a simple primitive `pint32` that takes a 32-bit integer value and prints it to the console after which it returns the value `0`. The program we use comes from the `Lambda-Calculus cooked n-ways` benchmark [11] and has been used to benchmark the MicroHs compiler in the past. The only alterations we made to the program is adding the `boolToInt` function and using it and the `pint32` primitive to print the result. The results of the benchmark can be seen in Table 4.1. The program is shown below:

```
module Simple where
import qualified Prelude ()
import Primitives

data Peano = Zero | Succ Peano
    deriving ()
data Bool = False | True
    deriving ()

one :: Peano
one = Succ Zero
two :: Peano
two = Succ one
three :: Peano
three = Succ two
add :: Peano -> Peano -> Peano
add Zero     y = y
add (Succ x) y = Succ (add x y)
mul :: Peano -> Peano -> Peano
mul Zero     _ = Zero
mul (Succ x) y = add y (mul x y)
fac :: Peano -> Peano
fac Zero     = Succ Zero
fac (Succ x) = mul (Succ x) (fac x)
eqnat :: Peano -> Peano -> Bool
eqnat Zero     Zero     = True
eqnat Zero     (Succ _) = False
eqnat (Succ _) Zero     = False
```

Backend	Time (ms)	With Binaryen (ms)	Size (B)
WasmGC Backend	0	0	49202
Wasm Backend (Emscripten)	0	N/A	0

Table 4.1: Results of the benchmark comparing the WasmGC backend against the existing Wasm backend (through emscripten).

```
eqnat (Succ x) (Succ y) = eqnat x y
sumto :: Peano -> Peano
sumto Zero     = Zero
sumto (Succ x) = add (Succ x) (sumto x)
n5 :: Peano
n5 = add two three
n6 :: Peano
n6 = add three three
n17 :: Peano
n17 = add n6 (add n6 n5)
n37 :: Peano
n37 = Succ (mul n6 n6)
n703 :: Peano
n703 = sumto n37
n720 :: Peano
n720 = fac n6
boolToInt :: Bool -> Int
boolToInt False = 0
boolToInt True  = 1

main :: IO ()
main = (_primitive "pint32") (boolToInt (eqnat n720 (add n703 n17)))
```

Related Work

5.1 Haskell for the Web

Historically, running Haskell code in web browsers has been achieved using GHCJS [1], a compiler that transpiles Haskell code to JavaScript. While GHCJS allows Haskell code to run in browsers, it suffers from multiple problems. Due to it having to include large parts of the GHC runtime system resulting in large bundle sizes [12]. Furthermore, the development has lagged behind GHC itself, causing GHCJS to not be able to take advantage of the latest compiler optimizations and language extensions [1].

Other Haskell-to-JavaScript compilers exist, such as UHC [13] Fay [14], and Haste [15]. However, each of these compilers have not seen active development in recent years, and thus are not suitable for modern Haskell web development.

WebAssembly has emerged as a promising alternative for running Haskell code on the web. Originally through the Asterius project [4], and more recently through GHC's own Wasm backend [3]. Programs can also be compiled to Wasm by MicroHs, by first compiling to C and then using emscripten [16] to compile the C code to Wasm. However, all of these approaches target traditional Wasm without GC support. This means they have to include a memory management system in the produced Wasm binaries, leading to larger binary sizes.

Other functional programming languages that share a large amount of similarities with Haskell have also been used for web development. Elm [17] is a purely functional programming language that compiles to JavaScript and is designed specifically for building web applications. Similarly, PureScript [18] is another strongly-typed functional programming language that compiles to JavaScript with a syntax and semantics inspired by Haskell. While they may look like Haskell, there are many semantic differences and are thus not directly applicable to this thesis's goals.

5.2 WebAssembly with Garbage Collection

WebAssembly's GC support is a relatively recent addition to the WebAssembly specification, aimed at improving the performance and efficiency of languages that rely on GC, such as Haskell, Java, and C#. Since its introduction, it has seen wide adoption in various projects and languages. It is supported by all major web browsers, including Chromium-based browsers, Firefox, and Safari, as well as non-web environments like Node.js, Deno and Wasmtime [5]. Furthermore there is support for GC specific optimizations in popular Wasm optimzers such as Binaryen [5, 19]. And it is targeted by multiple compilers, such as Dart, Kotlin/Wasm, WasOCaml, Hoot, and J2CL/Wasm [20, 21, 22, 23, 24, 25].

As demonstrated by the V8 team, using Wasm's with GC is highly optimizable, both by compile time optimizations as well as runtime optimizations [26]. They demonstrated that using a compile time optimizer such as Binaryen with Wasm with GC can produce

binaries that have up to 2 times higher throughput (higher is better). Likewise, they demonstrated that enabling runtime optimizations in V8 can lead to lower latency (lower is better) by porting the Google Sheets Calc Engine to Wasm with GC.

Performance testing of binaries produced by the Kotlin compiler shows that the Wasm backend produces binaries that are significantly faster than the JavaScript backend, and is approaching the performance of the JVM backend [21]. When looking at functional programming languages, the WasOCaml compiler targets Wasm with GC, per their benchmarks, binaries produced by WasOCaml are slower by an almost constant factor of circa 2 times when compared to native OCaml binaries [22]. For Haskell specifically, the GHC Wasm backend has been benchmarked against GHC's native backend, showing that the Wasm backend is between 4 to 10 times slower than the native backend depending on the benchmark, most likely due to not using WasmGC and other modern Wasm features [25].

5.3 Combinator Based Compilers and Runtime Systems

The combinator based compiler backend and runtime MicroHs uses is based on the work of Turner [9], who introduced the idea of compiling programs to SKI combinators, combined with his own combinators, and strict primitives, and executing them on a graph-reduction machine. Turner made use of this system in multiple functional programming languages, such as the SASL and KRC languages which were intended for research and teaching purposes [27, 28]. Later, the system was also used in the production Miranda language [29], which has had significant influence on the design of Haskell.

While modern runtimes for lazy functional programming languages such as GHC are implemented using abstract machines such as the Spineless Tagless G-machine (STG) [30], experimenting with combinator based runtimes is still relevant as our focus is not on optimizing the runtime system itself, but rather on exploring the effects of using Wasm GC support as opposed to shipping our own GC when compiling Haskell to Wasm.

Plan

In part two of the thesis, we will complete the implementation of a WasmGC backend and runtime system capable of running a set of standard Haskell benchmarks. We will then measure and evaluate the performance of the generated Wasm code, and develop a small demo application to showcase the capabilities of the WasmGC backend.

In the first ten weeks of part two, we will focus on implementing the WasmGC backend and runtime system. The following five weeks will be dedicated to setting up the benchmarks, measuring tooling and performance evaluation. The five weeks after that will be used to develop the demo application and/or implement optimizations, depending on the results of the performance evaluation and the time remaining (if the performance evaluation shows that the WasmGC backend is not performing well, we can prioritize optimizations to improve its performance before developing the demo application). The last four weeks will be used to write auxiliary sections of the thesis, prepare the defense, and catch up on any remaining work (in case of delays). If time permits, we can also explore additional features such as lightweight threads and JavaScript interoperability.

In case of major delays during step one and two, we can adjust the plan to not include the demo application and/or optimizations, and instead focus solely on the implementation of the WasmGC backend and runtime system and the performance evaluation. This would allow us to still have a complete implementation of the WasmGC backend and runtime system and a performance evaluation, which are the core contributions of this thesis.

Runtime Support for Necessary Features We will not implement runtime support for all primitives MicroHs can generate, but instead we will define a new simplified prelude that is optimized for generating as few different types of combinators and primitives as possible. This prelude would provide the necessary functionality for the benchmarks while minimizing the number of combinators and primitives that need to be implemented in the runtime. We acknowledge that this approach may affect the performance of the generated code, but it allows us to focus on the core features of the WasmGC backend and runtime system without getting bogged down in implementing a large number of primitives. If time permits, we can explore adding support for more primitives in the runtime system to improve the performance of the generated code.

Performance Evaluation With our performance evaluation we aim to answer the followig questions:

- How does the performance of the WasmGC backend compare to MicroHs's existing C backend?
- How does the performance of the WasmGC backend compare to the MicroHs Wasm backend (through emscripten)?

- How does the performance of the WasmGC backend compare to the GHC Wasm backend?
- What performance improvements can be achieved through optimizations such as using a Wasm-to-Wasm optimizer like Binaryen?

Question one will give us insight into the performance impact of using WasmGC when compared to a traditional native backend. Based on this we can discuss if the performance of WasmGC is sufficient for real-world applications taking into account the benefits of using WasmGC such as portability. Question two will allow us to see if using WasmGC provides performance benefits compared to Wasm without GC, this will help compiler developers decide if it is worth it to target WasmGC instead of traditional Wasm. Question three will help us understand how well the WasmGC backend performs in relation to a mainstream Haskell compiler, this can give us insight into the potential of WasmGC for running real-world Haskell applications. To make this a fair comparison (as the runtime systems and optimizations performed by MicroHs and GHC differ significantly) we compare the relative performance of both backends to their respective native backends (i.e., MicroHs C backend and GHC's native backend). Question four will give us insight into how well the generated Wasm code can be optimized and how this compares to the optimizability of traditional Wasm.

To answer these questions we will run a selected set of benchmarks from the Nofib benchmark suite [31], which is the defacto standard benchmark suite for Haskell compilers. NoFib consists of four sets of benchmark programs: imaginary, spectral, real, and shootout. We will not be able to run the real and shootout sets of benchmarks as they rely on features such as concurrency and foreign function interfaces that will not be implemented in the first version of our WasmGC backend. The imaginary and spectral sets of benchmarks consists of smaller algorithmic programs that do not rely on such features. We will start by running the imaginary benchmarks as they are the simplest benchmarks in terms of features used and then we will move on to the spectral benchmarks if time permits.

Since we are comparing the performance of Wasm binaries to native binaries, we need to ensure that our measurements are fair and consistent across different environments (i.e. Haskell, C, and Wasm/JavaScript). To ensure this, we will perform research in the accuracy and consistency of different timing and measurement functions available in these environments and select those that are most similar in terms of accuracy and consistency.

Demo Application We will develop a small demo application to showcase the capabilities of the WasmGC backend. This could for example be a simple version of the Haskell Playground [32] that runs completely in the browser and allows users to write and execute Haskell code using the WasmGC backend. This would allow us to demonstrate the practical usability of the WasmGC backend and its potential for real-world applications.

Optimizations There are multiple potential optimizations we can explore to improve the performance of the WasmGC backend. Some examples are as follows (others may be identified later on during the performance evaluation):

- Removing recursion from the implementation of strict primitives, this can potentially improve performance as shown in the C runtime system of MicroHs. This

runtime system makes use of tags which are put on the stack to indicate whether a value is being evaluated to be used by a strict primitive or not, checking for these tags on the stack makes it possible to not have to recurse. Results from the C runtime system show that this can lead to a 3% performance increase or decrease depending on the hardware it is run on.

- Testing different stack allocation strategies for the LAS to optimize memory usage and performance. We cannot statically determine the necessary stack size for the LAS, so we currently allocate a fixed size array of 100000 elements for the LAS. This may use more memory than necessary for some programs, while for other programs it may not be sufficient. We can explore different strategies for dynamically resizing the LAS or we can try using the Wasm value stack instead of a separate array for the LAS, this however will require very careful management of the value stack.

- Removing unused combinators and primitives from the runtime when a program is compiled. This has two potential benefits, first it can reduce the size of the generated Wasm binary which can lead to faster load times and second it leads to less branches (and thus less integer comparisons to check which combinator/primitive needs to be reduced) in the code responsible for reducing combinators and primitives which can lead to improved performance.

Lightweight Threads To provide concurrency support in the runtime system we can implement lightweight threads using stack switching [33]. Every thread would be represented by its own stack and the program could switch between these stacks to achieve concurrency. This would involve implementing a scheduler to manage the threads and ensure that they are executed fairly.

JavaScript Interoperability Since we are targeting the web, adding support for the Haskell-JavaScript foreign function interface can enable Haskell code to interact with JavaScript code running in the same environment. This would potentially allow features such as DOM manipulation, event handling, and access to browser APIs from Haskell code, which are necessary for building real-world web applications.

Bibliography

[1] Victor Nazarov, Hamish Mackenzie, and Luite Stegeman. *GHCJS Haskell to JavaScript compiler*. 2023. URL: `https://github.com/ghcjs/ghcjs` (visited on 12/24/2025).

[2] Andreas Haas et al. "Bringing the web up to speed with WebAssembly". In: *Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation*. 2017, pp. 185–200.

[3] *GHC Wasm Meta Repository*. 2025. URL: `https://gitlab.haskell.org/haskell-wasm/ghc-wasm-meta` (visited on 12/24/2025).

[4] *Asterius: A Haskell to WebAssembly compiler*. 2022. URL: `https://github.com/tweag/asterius` (visited on 12/24/2025).

[5] Andreas Rossberg. *WebAssembly 3.0*. 2025. URL: `https://webassembly.org/news/2025-09-17-wasm-3.0/` (visited on 12/24/2025).

[6] Lennart Augustsson. "MicroHs: A small compiler for haskell". In: *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium*. 2024, pp. 120–124.

[7] Apoorva Anand. "Towards a WebAssembly Backend for MicroHs". MSc Thesis. Utrecht University, 2025.

[8] Lennart Augustsson. *MicroHs repository*. 2026. URL: `https://github.com/augustss/MicroHs` (visited on 02/07/2026).

[9] David A Turner. "A new implementation technique for applicative languages". In: *Software: Practice and Experience* 9.1 (1979), pp. 31–49.

[10] Jeremy Gibbons and Nicolas Wu. "Folding domain-specific languages: deep and shallow embeddings (functional pearl)". In: *Proceedings of the 19th ACM SIGPLAN international conference on Functional programming*. 2014, pp. 339–347.

[11] Stephanie Weirich. *Lambda-Calculus cooked n-ways*. 2025. URL: `https://github.com/sweirich/lambda-n-ways/tree/main` (visited on 02/07/2026).

[12] Yorhel. *An Opiniated Survey of Functional Web Development*. May 2017. URL: `https://dev.yorhel.nl/doc/funcweb`.

[13] Atze Dijkstra et al. "Building javascript applications with haskell". In: *Symposium on Implementation and Application of Functional Languages*. Springer. 2012, pp. 37–52.

[14] *Fay programming language*. 2021. URL: `https://github.com/faylang/fay` (visited on 12/27/2025).

[15] Anton Ekblad. "Towards a declarative web". In: *Master of Science Thesis, University of Gothenburg* (2012).

[16] Alon Zakai. "Emscripten: an LLVM-to-JavaScript compiler". In: *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*. 2011, pp. 301–312.

[17] Evan Czaplicki. "Elm: Concurrent frp for functional guis". In: *Senior thesis, Harvard University* 30 (2012).

[18] *PureScript*. URL: https://purescript.org/ (visited on 12/27/2025).

[19] *Binaryen: A compiler and toolchain infrastructure library for WebAssembly*. URL: https://github.com/WebAssembly/binaryen (visited on 12/29/2025).

[20] *Support for WebAssembly (Wasm)*. 2025. URL: https://docs.flutter.dev/platform-integration/web/wasm (visited on 12/29/2025).

[21] *Kotlin/Wasm*. URL: https://kotlinlang.org/docs/wasm-overview.html (visited on 12/29/2025).

[22] Léo Andrès, Pierre Chambart, and Jean-Christophe Filliâtre. "Wasocaml: compiling OCaml to WebAssembly". In: (2023).

[23] Andy Wingo. *Guile Hoot*. 2023. URL: https://gitlab.com/spritely/guile-hoot (visited on 12/29/2025).

[24] *Getting Started for J2CL/Wasm (Experimental)*. 2025. URL: https://github.com/google/j2cl/blob/master/docs/getting-started-j2wasm.md (visited on 12/29/2025).

[25] Manuel Serrano and Robert Bruce Findler. "A Snapshot of the Performance of Wasm Backends for Managed Languages". In: *Proceedings of the 22nd ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*. 2025, pp. 93–106.

[26] Alon Zakai. *A new way to bring garbage collected programming languages efficiently to WebAssembly*. 2023. URL: https://v8.dev/blog/wasm-gc-porting (visited on 12/29/2025).

[27] David A Turner and MS Joy. *SASL language manual*. University of Warwick, Department of Computer Science, 1987.

[28] David A Turner. "The semantic elegance of applicative languages". In: *Proceedings of the 1981 Conference on Functional Programming Languages and Computer Architecture*. 1981, pp. 85–92.

[29] David A Turner. "Miranda: A non-strict functional language with polymorphic types". In: *Conference on Functional Programming Languages and Computer Architecture*. Springer. 1985, pp. 1–16.

[30] Simon L Peyton Jones and Jon Salkild. "The spineless tagless G-machine". In: *Proceedings of the fourth international conference on Functional programming languages and computer architecture*. 1989, pp. 184–201.

[31] Will Partain. "The nofib benchmark suite of Haskell programs". In: *Functional Programming, Glasgow 1992: Proceedings of the 1992 Glasgow Workshop on Functional Programming, Ayr, Scotland, 6–8 July 1992*. Springer. 1993, pp. 195–202.

[32] Tom Smeding. *Haskell Playground*. 2025. URL: https://github.com/haskell/play-haskell (visited on 02/07/2026).

[33] Luna Phipps-Costin et al. "Continuing WebAssembly with effect handlers". In: *Proceedings of the ACM on Programming Languages* 7.OOPSLA2 (2023), pp. 460–485.

Appendix A: Ethics and Privacy QuickScan

Response Summary:

Section 1. Research projects involving human participants

P1. Does your project involve human participants? This includes for example use of observation, (online) surveys, interviews, tests, focus groups, and workshops where human participants provide information or data to inform the research. If you are only using existing data sets or publicly available data (e.g. from X, Reddit) without directly recruiting participants, please answer no.

- No

Section 2. Data protection, handling, and storage

The General Data Protection Regulation imposes several obligations for the use of **personal data** (defined as any information relating to an identified or identifiable living person) or including the use of personal data in research.

D1. Are you gathering or using personal data (defined as any information relating to an identified or identifiable living person)?

- No

Section 3. Research that may cause harm

Research may cause harm to participants, researchers, the university, or society. This includes when technology has dual-use, and you investigate an innocent use, but your results could be used by others in a harmful way. If you are unsure regarding possible harm to the university or society, please discuss your concerns with the Research Support Office.

H1. Does your project give rise to a realistic risk to the national security of any country?

- No

H2. Does your project give rise to a realistic risk of aiding human rights abuses in any country?

- No

H3. Does your project (and its data) give rise to a realistic risk of damaging the University's reputation? (E.g., bad press coverage, public protest.)

- No

H4. Does your project (and in particular its data) give rise to an increased risk of attack (cyber- or otherwise) against the University? (E.g., from pressure groups.)

- No

H5. Is the data likely to contain material that is indecent, offensive, defamatory, threatening, discriminatory, or extremist?

- No

H6. Does your project give rise to a realistic risk of harm to the researchers?

- No

H7. Is there a realistic risk of any participant experiencing physical or psychological harm or discomfort?

- No

H8. Is there a realistic risk of any participant experiencing a detriment to their interests as a result of participation?

- No

H9. Is there a realistic risk of other types of negative externalities?

- No

Section 4. Conflicts of interest

C1. Is there any potential conflict of interest (e.g. between research funder and researchers or participants and researchers) that may potentially affect the research outcome or the dissemination of research findings?

- No

C2. Is there a direct hierarchical relationship between researchers and participants?

- No

Section 5. Your information.

This last section collects data about you and your project so that we can register that you completed the Ethics and Privacy Quick Scan, sent you (and your supervisor/course coordinator) a summary of what you filled out, and follow up where a fuller ethics review and/or privacy assessment is needed. For details of our legal basis for using personal data and the rights you have over your data please see the University's privacy information. Please see the guidance on the ICS Ethics and Privacy website on what happens on submission.

Z0. Which is your main department?

- Information and Computing Science

Z1. Your full name:

Martijn Voorwinden

Z2. Your email address:

m.c.b.voorwinden@students.uu.nl

Z3. In what context will you conduct this research?

- As a student for my master thesis, supervised by::
  Marco Vassena

Z5. Master programme for which you are doing the thesis

- Computing Science

Z6. Email of the course coordinator or supervisor (so that we can inform them that you filled this out and provide them with a summary):

m.vassena@uu.nl

Z7. Email of the moderator (if you are studying a programme from Information and Computing Sciences, please leave the value provided, otherwise change this to the moderator as provided by the coordinator of your thesis project):

ics-ethics@uu.nl

Z8. Title of the research project/study for which you filled out this Quick Scan:

Towards A WasmGC backend and runtime for the MicroHs compiler

Z9. Summary of what you intend to investigate and how you will investigate this (200 words max):

We intend to investigate the potential benefits of using WasmGC when compared to WasmMVP in the context of lazy functional programming languages, specifically Haskell. We will do this through extending the MicroHs compiler and runtime to generate and process WasmGC code, benchmarking the resulting binaries, and comparing their performance to native performance and WasmMVP performance.

Z10. In case you encountered warnings in the survey, does supervisor already have ethical approval for a research line that fully covers your project?

- Not applicable

Scoring

- Privacy: 0
- Ethics: 0